Embedded Linux development environment



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Embedded Linux solutions

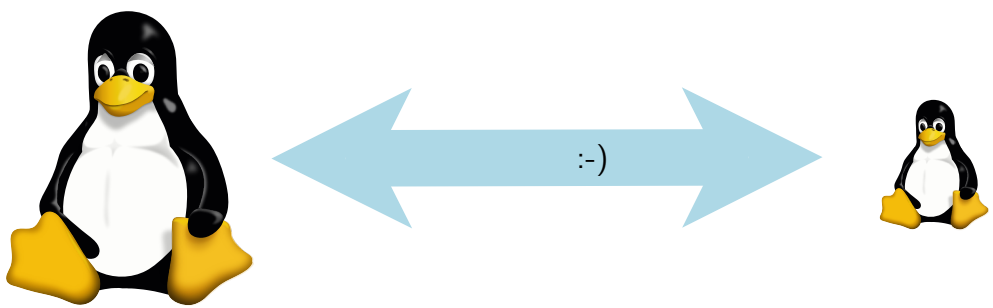
- Two ways to switch to embedded Linux
  - Use **solutions provided and supported by vendors** like MontaVista, Wind River or TimeSys. These solutions come with their own development tools and environment. They use a mix of open-source components and proprietary tools.
  - Use **community solutions**. They are completely open, supported by the community.
- In Bootlin training sessions, we do not promote a particular vendor, and therefore use community solutions
  - However, knowing the concepts, switching to vendor solutions will be easy



OS for Linux development

We strongly recommend to use GNU/Linux as the desktop operating system to embedded Linux developers, for multiple reasons.

- All community tools are developed and designed to run on Linux. Trying to use them on other operating systems (Windows, macOS) will lead to trouble.
- As Linux also runs on the embedded device, all the knowledge gained from using Linux on the desktop will apply similarly to the embedded device.
- If you are stuck with a Windows desktop, at least you should use GNU/Linux in a virtual machine (such as VirtualBox which is open source), though there could be a small performance penalty. With Windows 10/11, you can also run your favorite native Linux distro through Windows Subsystem for Linux (WSL2)





Desktop Linux distribution

- **Any good and sufficiently recent Linux desktop distribution** can be used for the development workstation
  - Ubuntu, Debian, Fedora, openSUSE, Arch Linux, etc.
- We have chosen Ubuntu, derived from Debian, as it is a **widely used and easy to use** desktop Linux distribution.
- The Ubuntu setup on the training laptops has intentionally been left untouched after the normal installation process. Learning embedded Linux is also about learning the tools needed on the development workstation!

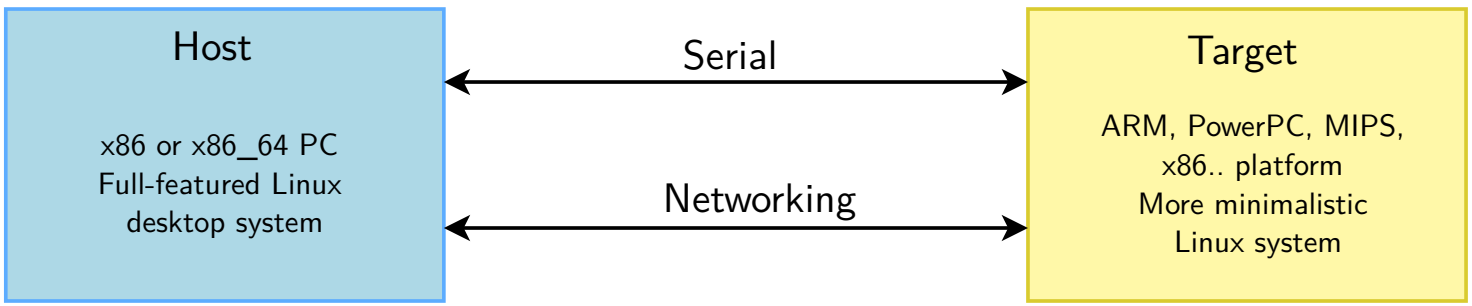


Image credits



Host vs. target

- When doing embedded development, there is always a split between
  - The *host*, the development workstation, which is typically a powerful PC
  - The *target*, which is the embedded system under development
- They are connected by various means: almost always a serial line for debugging purposes, frequently a networking connection, sometimes a JTAG interface for low-level debugging

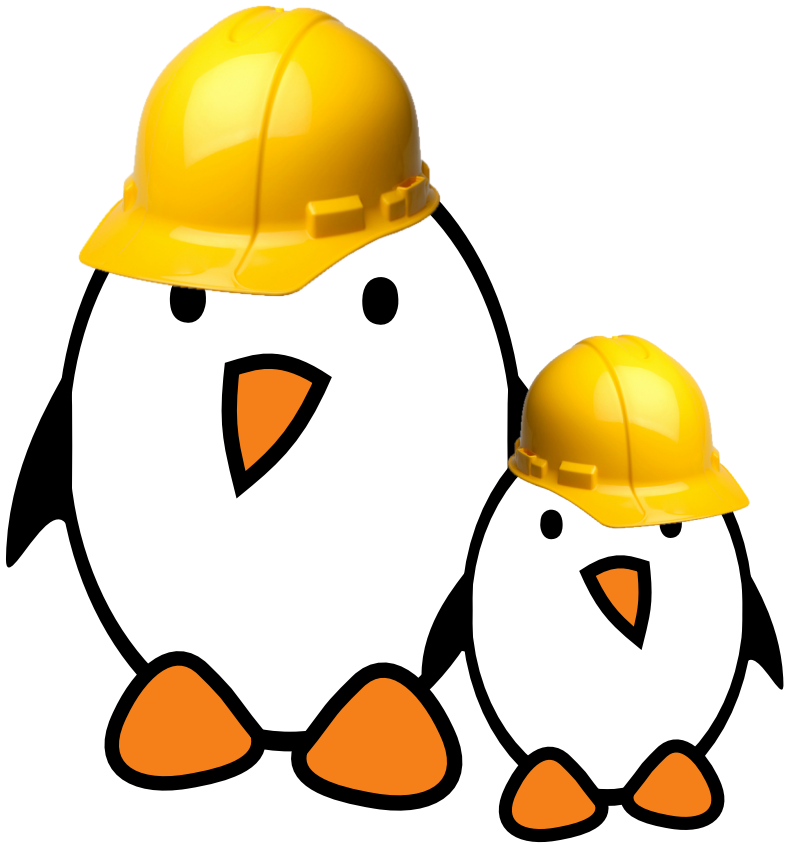




Serial line communication program

- An essential tool for embedded development is a serial line communication program, like *HyperTerminal* in Windows.
- There are multiple options available in Linux: *Minicom*, *Picocom*, *Gtkterm*, *Putty*, *screen*, *tmux* and the new *tio* (https://github.com/tio/tio).
- In this training session, we recommend using the simplest of them: *Picocom*
  - Installation with `sudo apt install picocom`
  - Run with `picocom -b BAUD_RATE /dev/SERIAL_DEVICE`.
  - Exit with `[Ctrl][a] [Ctrl][x]`
- `SERIAL_DEVICE` is typically
  - `ttyUSBx` for USB to serial converters
  - `ttySx` for real serial ports
- Most frequent command: `picocom -b 115200 /dev/ttyUSB0`

Practical lab - Training Setup



Prepare your lab environment

- Download and extract the lab archive